Date:

Q2 Coin Change :

Approach ,

→ To solve the problem we have created a function which calculates the no. of coins two times once ~~int~~ including the current coin and once excluding it. This is done using recursion.

→ To optimize the program a two dimensional dp is used with current index i & sum as the two dimensions.

Pseudocode ,

```
class Solution {
    public static int Solve (int[] arr, int sum,
                                int i , int[][] dp) {
        if (sum == 0) {
            return 0;
        }
        if (sum < 0 || i == arr.length) {
            return 99999;
        }
        if (dp[i] [sum] != -1) {
            return dp[i][sum];
        }
        int take = 1 + solve (arr, sum-arr[i],i, dp);
        int not_take = Solve (arr, sum, i+1, dp);
        dp[i][sum] = Math.min (take, not_take);
        return dp[i][sum];
    }
```

Date:

```
public int coinchange(int [] coins, int amount){

    int[][] dp = new int [coins.length]
                    [amount + 1];
    for (int i = 0; i < coins.length; i++){
        for (int j = 0; j < amount+1 ; j++) {
            dp[i][j] = -1;
        }
    }

    int ans = Solve (coins, amount,
                    0, dp);
    if (ans >= 99999) return -1;
    return ans;
}
}
```